PYTHON ACTIVITY-BASED LEARNING

Documentation for 10 Selected Case Studies

Selected Case Studies	1, 4, 9, 13, 17, 21, 37, 47, 52, 89
Course	Python Programming
Student Name	N. Srujan
Roll Number / Section	26000111 2
Department / College	CSE / Dr.rvr.nri institute of technology
Academic Year	2026-2030

Note: This is a worked sample based on the case-study titles and requirements in the supplied assignment list. Replace illustrative thresholds, rates, and sample data with any exact rules given by your faculty. Run each program yourself and capture your actual output screenshots before submission.

Contents

Case Study 1: University Scholarship Allocation

Case Study 4: Employee Payroll and Deduction Analysis

Case Study 9: Multi-Currency Travel Budget Analysis

Case Study 13: ATM Transaction Sequence Simulation

Case Study 17: Student Performance Trend Analysis

Case Study 21: Traffic Violation Penalty Calculator

Case Study 37: Nested Loop Calendar Generator

Case Study 47: Sports Tournament Ranking System

Case Study 52: Hackathon Team Formation and Ranking

Case Study 89: University Hostel Room Allocation

Documentation format used for each case: problem statement, objectives, assumptions, algorithm, pseudocode, flowchart, Python source code, test cases, sample output, and conclusion.

Case Study 1: University Scholarship Allocation

1. Problem Statement

Evaluate applicants using aggregate marks, subject-wise minimum scores, attendance, and family income; rank eligible applicants and resolve ties using predefined rules.

2. Objectives

• Check each applicant against all eligibility rules.

• Calculate aggregate percentage and rank eligible applicants.

• Use aggregate percentage, then attendance, as tie-break criteria.

3. Assumptions

• Marks are entered as percentages from 0 to 100 for three subjects.

• Eligibility requires aggregate >= 80, each subject >= 70, attendance >= 75%, and annual family income <= Rs. 300000.

• Ranking uses aggregate percentage first, then attendance; sample data is illustrative.

4. Algorithm

1. Read applicant records: name, three subject marks, attendance, and income.

2. Calculate aggregate as the average of the three marks.

3. Check every eligibility condition.

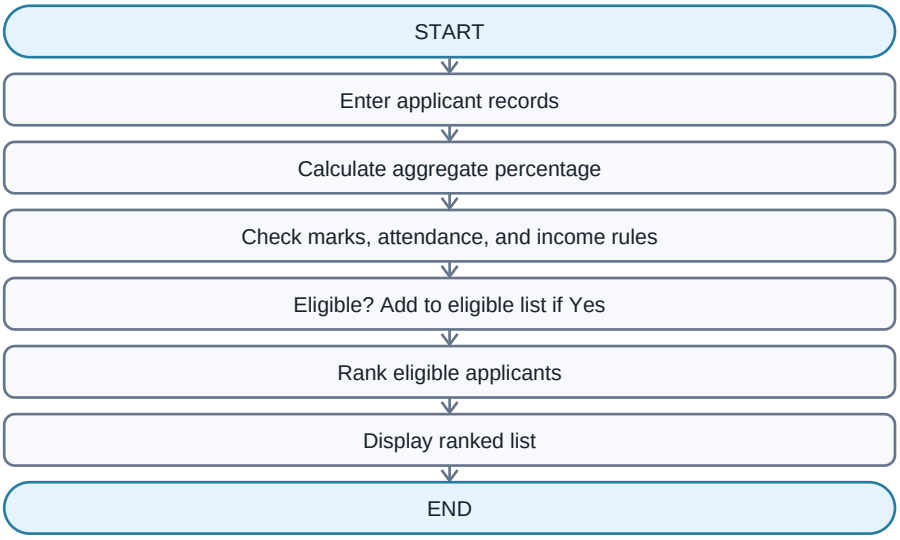
4. Store eligible applicants with aggregate and attendance.

5. Sort by aggregate descending, then attendance descending; display results.

5. Pseudocode

```
START
Create applicant list
FOR each applicant
  aggregate = average of three marks
  IF aggregate >= 80 AND each mark >= 70
     AND attendance >= 75 AND income <= 300000
       add applicant to eligible list
Sort eligible list by aggregate, then attendance
DISPLAY eligible ranking
END FOR
END
```

6. Flowchart



7. Python Source Code

```
applicants = [
    ("Asha", 88, 82, 91, 92, 250000),
    ("Bala", 78, 85, 80, 90, 220000),
```

```
    ("Charan", 92, 90, 88, 70, 200000),
    ("Divya", 84, 83, 82, 86, 280000)
]

eligible = []
for name, m1, m2, m3, attendance, income in applicants:
    aggregate = (m1 + m2 + m3) / 3
    if (aggregate >= 80 and m1 >= 70 and m2 >= 70 and m3 >= 70
            and attendance >= 75 and income <= 300000):
        eligible.append((name, aggregate, attendance))

eligible.sort(key=lambda item: (item[1], item[2]), reverse=True)
print("Eligible scholarship applicants")
for rank, student in enumerate(eligible, start=1):
    print(rank, student[0], round(student[1], 2), student[2])
```

8. Test Cases

Test / Input	Expected result
Asha	Eligible; aggregate 87.00
Bala	Eligible if all subject marks meet minimum; aggregate 81.00
Charan	Not eligible; attendance 70%
Divya	Eligible; aggregate 83.00

9. Sample Output

```
Eligible scholarship applicants
1 Asha 87.0 92
2 Divya 83.0 86
3 Bala 81.0 90
```

10. Conclusion

The program applies multiple scholarship conditions and ranks only eligible applicants. The thresholds and sample records should be changed to match the faculty's prescribed rules.

Case Study 4: Employee Payroll and Deduction Analysis

1. Problem Statement

Calculate gross salary from basic pay, allowances, overtime, and incentives; apply progressive deductions and generate a formatted salary statement.

2. Objectives

• Calculate HRA and DA from basic salary.

• Calculate overtime pay and gross salary.

• Apply illustrative progressive deductions and display net salary.

3. Assumptions

• HRA = 20% of basic salary; DA = 10% of basic salary.

• Overtime is paid at the entered overtime rate per hour.

• For demonstration only: deduction is 5% of gross salary up to Rs. 30000, 10% on the next Rs. 20000, and 15% on the remaining amount. These are not official tax rules.

4. Algorithm

1. Read employee name, basic salary, overtime hours, overtime rate, and incentive.

2. Calculate HRA, DA, and overtime pay.

3. Calculate gross salary.

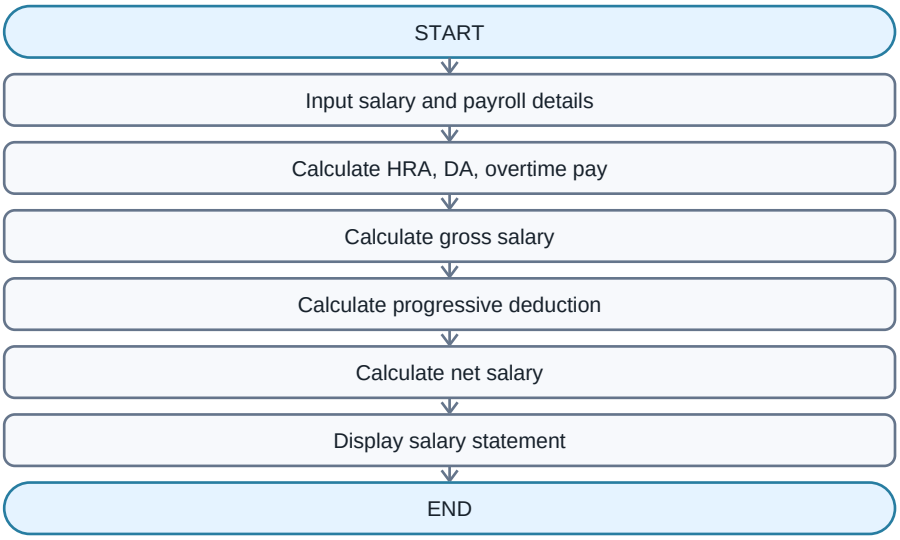
4. Calculate progressive deduction in slabs.

5. Subtract deduction from gross salary and print the salary statement.

5. Pseudocode

```
START
INPUT employee details
HRA = 0.20 * basic salary
DA = 0.10 * basic salary
overtime pay = overtime hours * overtime rate
gross = basic + HRA + DA + overtime pay + incentive
Calculate deduction using the stated slabs
net = gross - deduction
DISPLAY salary statement
END
```

6. Flowchart



7. Python Source Code

```
name = input("Employee name: ")
basic = float(input("Basic salary (Rs.): "))
```

```
overtime_hours = float(input("Overtime hours: "))
overtime_rate = float(input("Overtime rate per hour (Rs.): "))
incentive = float(input("Incentive (Rs.): "))

hra = basic * 0.20
da = basic * 0.10
overtime_pay = overtime_hours * overtime_rate
gross = basic + hra + da + overtime_pay + incentive

if gross <= 30000:
    deduction = gross * 0.05
elif gross <= 50000:
    deduction = 30000 * 0.05 + (gross - 30000) * 0.10
else:
    deduction = 30000 * 0.05 + 20000 * 0.10 + (gross - 50000) * 0.15

net = gross - deduction
print("\nSALARY STATEMENT")
print("Employee:", name)
print("HRA:", round(hra, 2), "DA:", round(da, 2))
print("Overtime pay:", round(overtime_pay, 2))
print("Gross salary:", round(gross, 2))
print("Deduction:", round(deduction, 2))
print("Net salary:", round(net, 2))
```

8. Test Cases

Test / Input	Expected result
Basic 30000, OT 5h x 200, incentive 2000	Gross Rs. 42000; deduction Rs. 2700; net Rs. 39300
Basic 20000, OT 0, incentive 0	Gross Rs. 26000; deduction Rs. 1300; net Rs. 24700
Basic 40000, OT 2h x 250, incentive 1000	Gross Rs. 53500; deduction Rs. 4025; net Rs. 49475

9. Sample Output

```
Employee: Rani
HRA: 6000.0 DA: 3000.0
Overtime pay: 1000.0
Gross salary: 42000.0
Deduction: 2700.0
Net salary: 39300.0
```

10. Conclusion

The program produces a salary statement by combining pay components and subtracting deductions. The deduction slabs are classroom assumptions and must be replaced if the assignment specifies different rules.

Case Study 9: Multi-Currency Travel Budget Analysis

1. Problem Statement

Convert travel expenses using supplied exchange rates, include conversion fees, compare spending with a budget, and identify the largest expense category.

2. Objectives

• Convert foreign currency expenses into INR.

• Add a conversion fee and calculate total spending.

• Compare the total with the budget and identify the largest category.

3. Assumptions

• Exchange rates are sample inputs, not live rates: USD Rs. 88, EUR Rs. 102, GBP Rs. 117, INR Rs. 1.

• A 2% conversion fee applies to non-INR expenses.

• Budget is Rs. 100000; category amounts are illustrative.

4. Algorithm

1. Store exchange rates and expense records.

2. For each expense, convert the amount to INR.

3. Apply 2% fee to foreign-currency expenses.

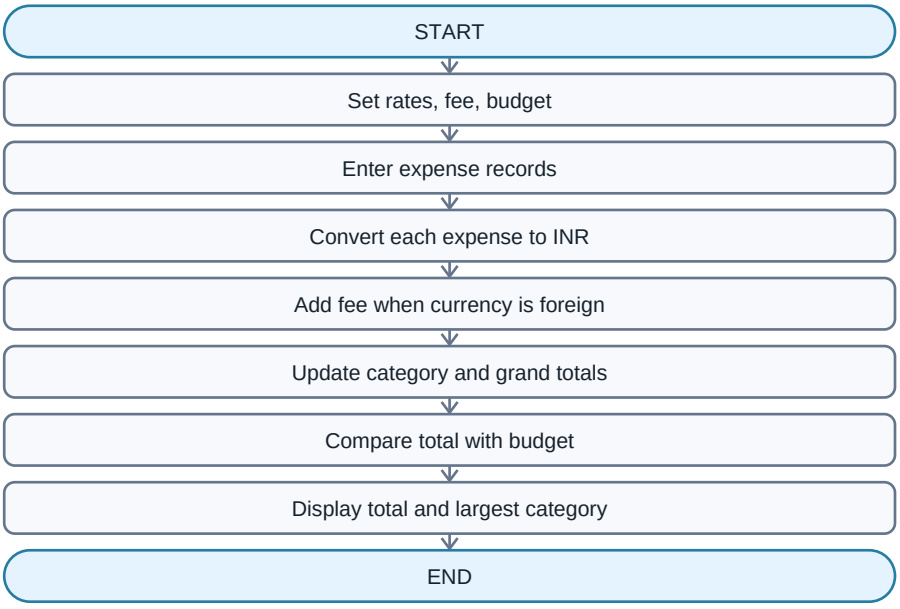
4. Add converted expenses to category totals and grand total.

5. Compare total with budget and display largest category.

5. Pseudocode

```
START
Set exchange rates and budget
Set total = 0; create category totals
FOR each expense
  converted = amount * exchange rate
  IF currency is not INR, add 2% fee
  Add converted amount to category total and total
Find category with maximum total
Compare total with budget
DISPLAY report
END
```

6. Flowchart



7. Python Source Code

```
rates = {"INR": 1, "USD": 88, "EUR": 102, "GBP": 117}
budget = 100000
expenses = [
    ("Hotel", 300, "USD"),
    ("Food", 120, "EUR"),
    ("Transport", 5000, "INR"),
    ("Shopping", 100, "GBP")
]

category_totals = {}
grand_total = 0
for category, amount, currency in expenses:
    converted = amount * rates[currency]
    if currency != "INR":
        converted = converted * 1.02
    category_totals[category] = converted
    grand_total += converted

largest = max(category_totals, key=category_totals.get)
print("Category totals in INR:")
for category, amount in category_totals.items():
    print(category, round(amount, 2))
print("Total spending:", round(grand_total, 2))
print("Budget:", budget)
print("Budget status:", "Within budget" if grand_total <= budget else "Over budget")
print("Largest category:", largest)
```

8. Test Cases

Test / Input	Expected result
Hotel 300 USD	Rs. 26928.00 including 2% fee
Food 120 EUR	Rs. 12484.80 including 2% fee
Total sample expenses	Rs. 56346.80 total spending

9. Sample Output

```
Category totals in INR:
Hotel 26928.0
Food 12484.8
Transport 5000
Shopping 11934.0
Total spending: 56346.8
Budget: 100000
Budget status: Within budget
Largest category: Hotel
```

10. Conclusion

The program converts each expense to INR, adds the fee where applicable, and compares the result with the budget. Exchange rates are illustrative and should be entered as supplied in the assignment.

Case Study 13: ATM Transaction Sequence Simulation

1. Problem Statement

Process deposits and withdrawals, maintain a simulated balance, enforce withdrawal limits, reject invalid transactions, and produce a transaction summary.

2. Objectives

• Update balance after each valid transaction.

• Reject invalid amounts and withdrawals that exceed balance or limit.

• Report final balance and transaction counts.

3. Assumptions

• Opening balance is Rs. 10000.

• Maximum withdrawal per transaction is Rs. 5000.

• Transactions are represented by D for deposit and W for withdrawal; amounts must be positive.

4. Algorithm

1. Set opening balance and withdrawal limit.

2. Read each transaction type and amount from the sample list.

3. Validate amount and transaction type.

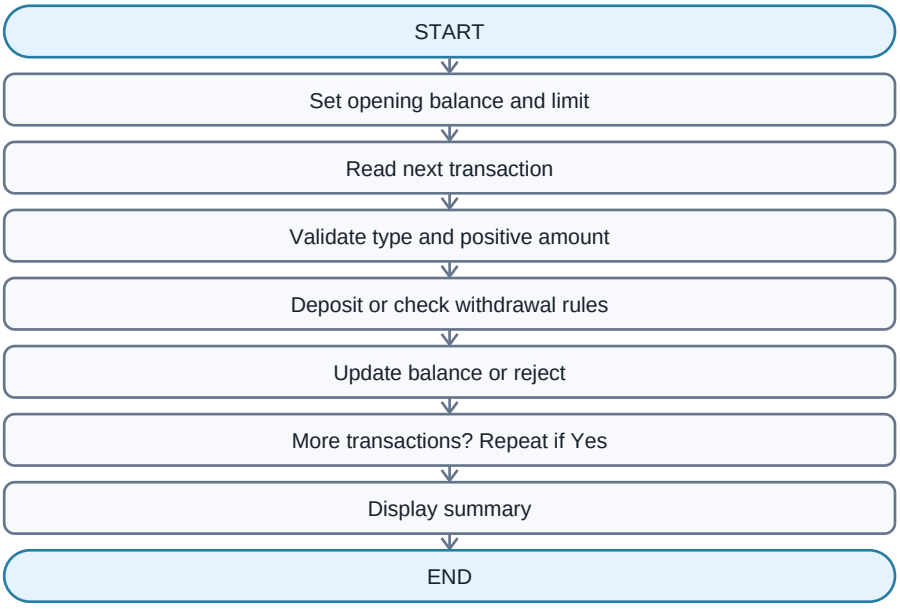
4. For a deposit, add amount; for withdrawal, check limit and balance.

5. Report accepted or rejected transaction and final balance.

5. Pseudocode

```
START
balance = 10000; limit = 5000
FOR each transaction
  IF amount <= 0, reject
  ELSE IF type is D, add amount
  ELSE IF type is W
    IF amount > limit OR amount > balance, reject
    ELSE subtract amount
  ELSE reject invalid type
Display final balance and summary
END
```

6. Flowchart



7. Python Source Code

```
balance = 10000
withdrawal_limit = 5000
transactions = [("D", 2000), ("W", 3000), ("W", 6000), ("W", 500), ("D", -100)]
accepted = 0
rejected = 0

for kind, amount in transactions:
    if amount <= 0:
        print("Rejected: amount must be positive", kind, amount)
        rejected += 1
    elif kind == "D":
        balance += amount
        accepted += 1
        print("Deposit accepted:", amount, "Balance:", balance)
    elif kind == "W":
        if amount > withdrawal_limit or amount > balance:
            print("Withdrawal rejected:", amount)
            rejected += 1
        else:
            balance -= amount
            accepted += 1
            print("Withdrawal accepted:", amount, "Balance:", balance)
    else:
        print("Rejected: invalid transaction type")
        rejected += 1

print("Final balance:", balance)
print("Accepted:", accepted, "Rejected:", rejected)
```

8. Test Cases

Test / Input	Expected result
D, 2000	Accepted; balance Rs. 12000
W, 6000	Rejected; exceeds per-transaction limit
W, -100	Rejected; amount must be positive

9. Sample Output

```
Deposit accepted: 2000 Balance: 12000
Withdrawal accepted: 3000 Balance: 9000
Withdrawal rejected: 6000
Withdrawal accepted: 500 Balance: 8500
Rejected: amount must be positive W -100
Final balance: 8500
Accepted: 3 Rejected: 2
```

10. Conclusion

The simulation demonstrates sequential transaction processing and validation. It is a classroom model and is not connected to a real bank account.

Case Study 17: Student Performance Trend Analysis

1. Problem Statement

Compare assessment scores across examinations, calculate improvements and declines, identify weak subjects, and classify performance using defined criteria.

2. Objectives

• Compare first and second assessment scores by subject.

• Calculate total-score change.

• Identify subjects below the minimum target and classify overall trend.

3. Assumptions

• Marks are out of 100.

• A subject is flagged as weak if the latest score is below 40.

• Overall trend is improving if the latest total is greater than the earlier total; unchanged totals are stable.

4. Algorithm

1. Store subject names and scores for two assessments.

2. Calculate each subject's change and identify weak latest scores.

3. Calculate earlier and latest totals.

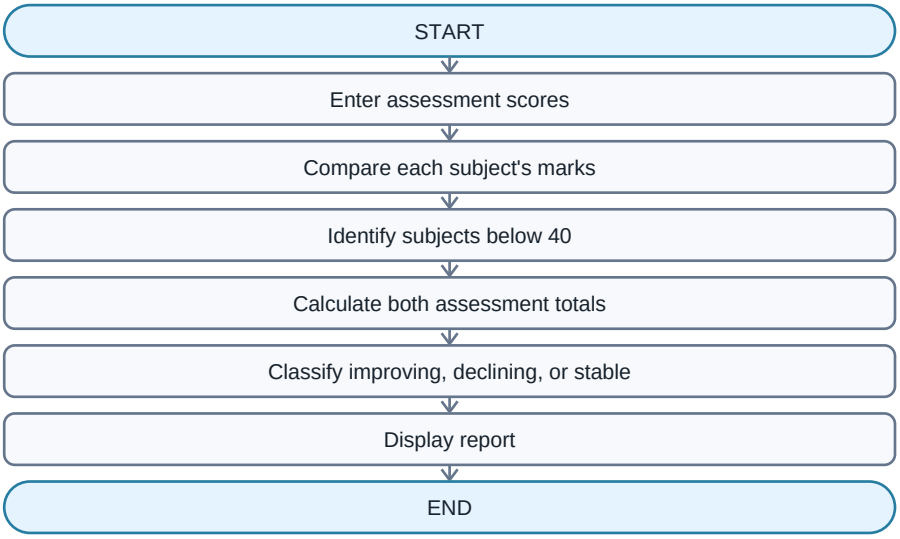
4. Classify the trend based on total comparison.

5. Display subject changes, weak subjects, and trend.

5. Pseudocode

```
START
Store subjects and two sets of marks
Set earlier_total = 0 and latest_total = 0
FOR each subject
  change = latest - earlier
  Update totals
  IF latest < 40, mark subject weak
Compare totals to classify trend
DISPLAY analysis
END
```

6. Flowchart



7. Python Source Code

```
subjects = ["Python", "Maths", "English", "BEEE"]
first = [65, 72, 58, 45]
latest = [78, 68, 64, 38]
```

```
first_total = 0
latest_total = 0
weak = []

print("Subject  First  Latest  Change")
for i in range(len(subjects)):
    change = latest[i] - first[i]
    first_total += first[i]
    latest_total += latest[i]
    print(subjects[i], first[i], latest[i], change)
    if latest[i] < 40:
        weak.append(subjects[i])

if latest_total > first_total:
    trend = "Improving"
elif latest_total < first_total:
    trend = "Declining"
else:
    trend = "Stable"

print("Earlier total:", first_total)
print("Latest total:", latest_total)
print("Trend:", trend)
print("Weak subjects:", weak if weak else "None")
```

8. Test Cases

Test / Input	Expected result
Python 65 to 78	Change +13
BEEE 45 to 38	Change -7; flagged weak
Totals 240 to 248	Overall trend improving

9. Sample Output

```
Subject  First  Latest  Change
Python 65 78 13
Maths 72 68 -4
English 58 64 6
BEEE 45 38 -7
Earlier total: 240
Latest total: 248
Trend: Improving
Weak subjects: ['BEEE']
```

10. Conclusion

The program compares two assessments and identifies changes by subject. The weak-score threshold and classification rule are assumptions that can be adjusted to the faculty's criteria.

Case Study 21: Traffic Violation Penalty Calculator

1. Problem Statement

Process multiple violations using vehicle category, offence type, repeat-offence status, and supplied fine rules; calculate total penalty and identify disqualification conditions.

2. Objectives

• Calculate penalties using a simple rule table.

• Apply an illustrative repeat-offence multiplier.

• Flag serious violations for review.

3. Assumptions

• Illustrative fines only: speeding Rs. 1000, signal-jumping Rs. 1500, no-helmet Rs. 500.

• A repeat offence doubles the base fine.

• Any signal-jumping offence is flagged for review in this sample model. These are not official legal rates or legal advice.

4. Algorithm

1. Store offence records with type and repeat status.

2. Look up base fine for each offence.

3. Double the fine for repeat offences.

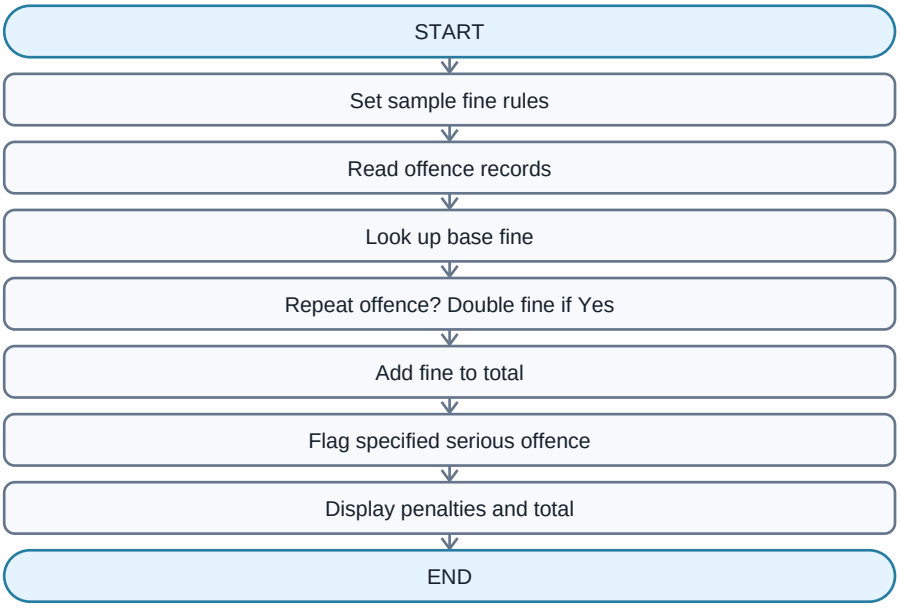
4. Add each fine to the total and flag signal-jumping records.

5. Display each penalty and total.

5. Pseudocode

```
START
Set fine table
total = 0
FOR each offence
  Find base fine
  IF repeat, fine = base fine * 2
  ELSE fine = base fine
  Add fine to total
  IF offence is signal-jumping, flag for review
DISPLAY itemized fines and total
END
```

6. Flowchart



7. Python Source Code

```
base_fines = {"speeding": 1000, "signal-jumping": 1500, "no-helmet": 500}
violations = [("speeding", False), ("no-helmet", True), ("signal-jumping", False)]
total = 0

for offence, repeat in violations:
    if offence not in base_fines:
        print("Unknown offence:", offence)
        continue
    fine = base_fines[offence]
    if repeat:
        fine = fine * 2
    total += fine
    print(offence, "repeat" if repeat else "first offence", "fine Rs.", fine)
    if offence == "signal-jumping":
        print("Flag: review applicable rules")

print("Total illustrative penalty: Rs.", total)
```

8. Test Cases

Test / Input	Expected result
speeding, first offence	Rs. 1000
no-helmet, repeat	Rs. 1000
signal-jumping, first offence	Rs. 1500 and flagged

9. Sample Output

```
speeding first offence fine Rs. 1000
no-helmet repeat fine Rs. 1000
signal-jumping first offence fine Rs. 1500
Flag: review applicable rules
Total illustrative penalty: Rs. 3500
```

10. Conclusion

The program demonstrates rule-based penalty calculation. Fine amounts and consequences are purely illustrative and must be replaced by the exact rules supplied for the assignment.

Case Study 37: Nested Loop Calendar Generator

1. Problem Statement

Generate a formatted monthly calendar using a supplied starting weekday and number of days, aligning weeks and validating the month length.

2. Objectives

• Print weekday headings and date positions.

• Use nested loops to create calendar rows.

• Reject invalid start-day and month-length inputs.

3. Assumptions

• Week starts on Sunday, represented by start_day = 0; Saturday is 6.

• Month length must be between 28 and 31 days.

• The month name is supplied by the user.

4. Algorithm

1. Read month name, number of days, and starting weekday number.

2. Validate days and starting weekday.

3. Print weekday headings and leading spaces.

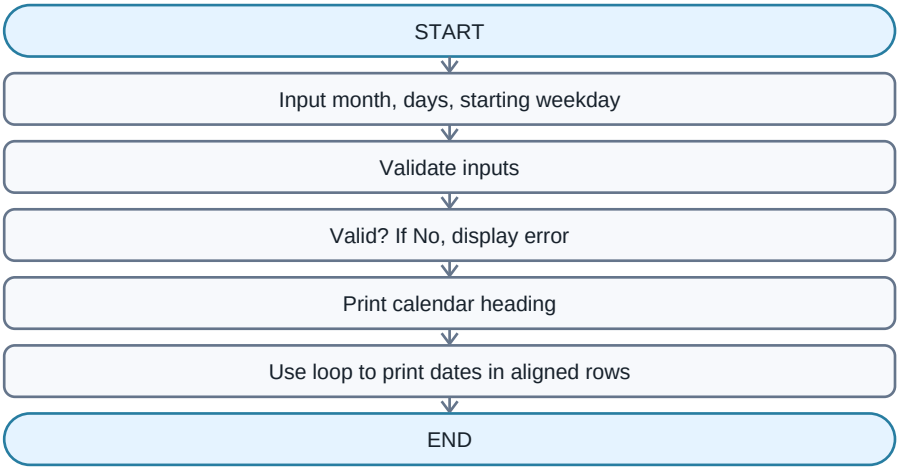
4. Use a loop for each day and start a new row after Saturday.

5. Print the final calendar.

5. Pseudocode

```
START
INPUT month, days, start_day
IF days not 28..31 OR start_day not 0..6, show error
ELSE print month and weekday headings
Print start_day blank positions
FOR day from 1 to days
  Print day in fixed-width field
  IF (start_day + day) modulo 7 = 0, new line
END FOR
END
```

6. Flowchart



7. Python Source Code

```
month = input("Month name: ")
days = int(input("Number of days (28-31): "))
start_day = int(input("Starting weekday (Sun=0 ... Sat=6): "))

if days < 28 or days > 31 or start_day < 0 or start_day > 6:
    print("Invalid calendar details")
else:
```

```
    print("\n", month)
    print("Su Mo Tu We Th Fr Sa")
    print("   " * start_day, end="")
    for day in range(1, days + 1):
        print(f"{day:2}", end=" ")
        if (start_day + day) % 7 == 0:
            print()
    print()
```

8. Test Cases

Test / Input	Expected result
days=31, start_day=0	Calendar starts with day 1 under Sunday
days=30, start_day=3	Three blank weekday positions before day 1
days=35	Invalid calendar details

9. Sample Output

```
Month name: April
Number of days (28-31): 30
Starting weekday (Sun=0 ... Sat=6): 3
April
Su Mo Tu We Th Fr Sa
          1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30
```

10. Conclusion

The program uses loops and weekday offsets to format a basic monthly calendar. It does not calculate real month lengths or leap years automatically; those values are supplied as input.

Case Study 47: Sports Tournament Ranking System

1. Problem Statement

Store match results and team scores, calculate points and tie-break values, and rank teams using a specified competition policy.

2. Objectives

• Award points for wins, draws, and losses.

• Calculate goal difference as a tie-break value.

• Display a sorted ranking table.

3. Assumptions

• A win earns 3 points, a draw earns 1, and a loss earns 0.

• Goal difference equals goals scored minus goals conceded.

• Ranking order is points descending, goal difference descending, then team name alphabetically.

4. Algorithm

1. Create team records with played matches, wins, draws, losses, goals for, and goals against.

2. Calculate points and goal difference for each team.

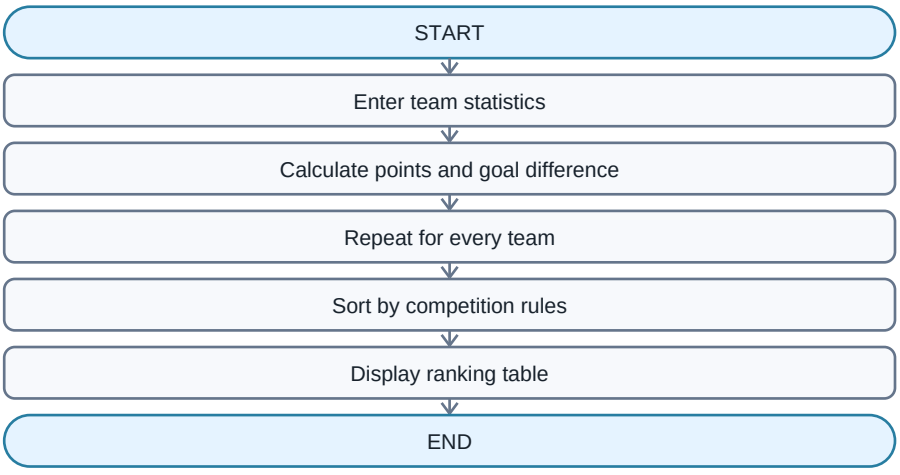
3. Sort teams by points, goal difference, then name.

4. Display ranked teams.

5. Pseudocode

```
START
Store team statistics
FOR each team
  points = wins * 3 + draws
  goal_difference = goals_for - goals_against
Sort by points descending, goal difference descending, name
DISPLAY rankings
END
```

6. Flowchart



7. Python Source Code

```
teams = [
    ("Falcons", 3, 1, 0, 10, 4),
    ("Tigers", 3, 0, 1, 8, 5),
    ("Eagles", 1, 2, 1, 6, 6),
    ("Wolves", 1, 1, 2, 5, 8)
]
ranking = []
for name, wins, draws, losses, gf, ga in teams:
    points = wins * 3 + draws
    goal_difference = gf - ga
    ranking.append((name, points, goal_difference, gf))
```

```
ranking.sort(key=lambda team: (-team[1], -team[2], team[0]))
print("Rank Team Points GD")
for rank, team in enumerate(ranking, start=1):
    print(rank, team[0], team[1], team[2])
```

8. Test Cases

Test / Input	Expected result
Falcons: 3 wins, 1 draw	10 points
Tigers: 3 wins	9 points
Eagles: 1 win, 2 draws	5 points

9. Sample Output

```
Rank Team Points GD
1 Falcons 10 6
2 Tigers 9 3
3 Eagles 5 0
4 Wolves 4 -3
```

10. Conclusion

The program calculates league points and applies tie-break rules to produce a ranking. The sample assumes the team statistics are already verified and consistent.

Case Study 52: Hackathon Team Formation and Ranking

1. Problem Statement

Analyse student scores and team compositions, validate team sizes, aggregate round-wise scores, and resolve tied rankings using predefined rules.

2. Objectives

• Check that each team has an allowed number of members.

• Add round scores to calculate team totals.

• Rank valid teams and report invalid team sizes.

3. Assumptions

• Each team must contain 2 to 4 members.

• Each team has three round scores.

• Ranking is by total score descending; team name is the tie-break.

4. Algorithm

1. Store team name, member list, and three round scores.

2. Validate team size.

3. For valid teams, calculate total score and store the result.

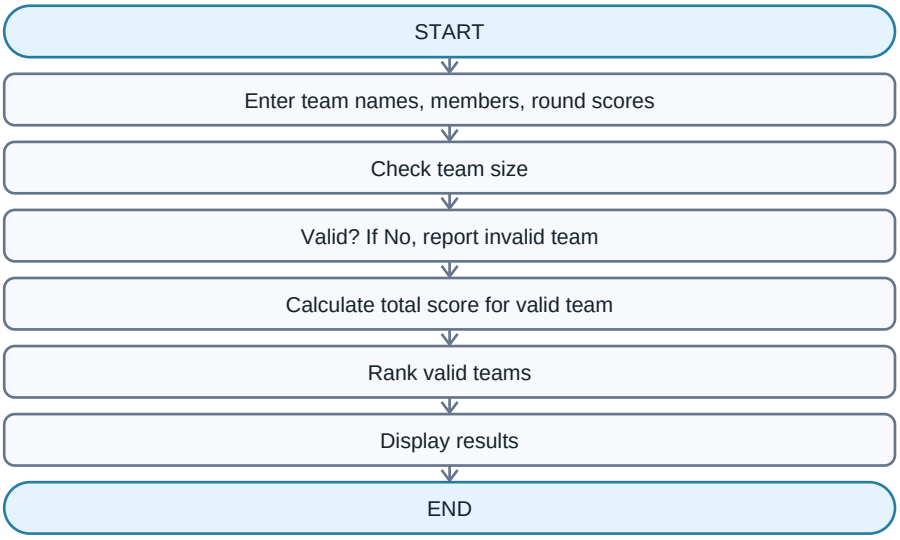
4. Sort valid teams by total score and team name.

5. Display ranked valid teams and invalid team notices.

5. Pseudocode

```
START
Store team records
FOR each team
  IF member count < 2 OR > 4, mark invalid
  ELSE total = sum of round scores; store team
Sort valid teams by total descending, then name
DISPLAY ranking and invalid teams
END
```

6. Flowchart



7. Python Source Code

```
teams = [
    ("ByteBuilders", ["Asha", "Bala", "Charan"], [80, 85, 90]),
    ("CodeCrew", ["Divya", "Eshan"], [90, 82, 88]),
    ("SoloCoder", ["Farah"], [95, 90, 92]),
    ("LogicLoop", ["Gita", "Hari", "Ira", "Jay"], [78, 84, 80])
]
```

```
valid = []
for name, members, scores in teams:
    if len(members) < 2 or len(members) > 4:
        print("Invalid team size:", name, "-", len(members), "members")
    else:
        total = sum(scores)
        valid.append((name, len(members), total))

valid.sort(key=lambda team: (-team[2], team[0]))
print("\nTeam ranking")
for rank, team in enumerate(valid, start=1):
    print(rank, team[0], "members:", team[1], "total:", team[2])
```

8. Test Cases

Test / Input	Expected result
ByteBuilders scores 80,85,90	Total 255; valid 3 members
SoloCoder has 1 member	Invalid team size
CodeCrew scores 90,82,88	Total 260; ranks above ByteBuilders

9. Sample Output

```
Invalid team size: SoloCoder - 1 members

Team ranking
1 CodeCrew members: 2 total: 260
2 ByteBuilders members: 3 total: 255
3 LogicLoop members: 4 total: 242
```

10. Conclusion

The program validates team composition, totals round scores, and ranks eligible teams. Team-size limits and ranking criteria are sample rules.

Case Study 89: University Hostel Room Allocation

1. Problem Statement

Allocate students to rooms according to capacity, room eligibility, and grouping constraints; detect duplicate assignments and calculate unallocated demand.

2. Objectives

• Allocate eligible students to available room capacity.

• Avoid assigning the same student more than once.

• Report students who cannot be allocated.

3. Assumptions

• Rooms have fixed capacities and are processed in listed order.

• Students have a simple eligibility flag: True means eligible for the room type in this sample.

• Each student needs one bed; allocation is first-fit and does not optimize preferences.

4. Algorithm

1. Store room names and capacities.

2. Store student names and eligibility.

3. Track remaining capacity for each room.

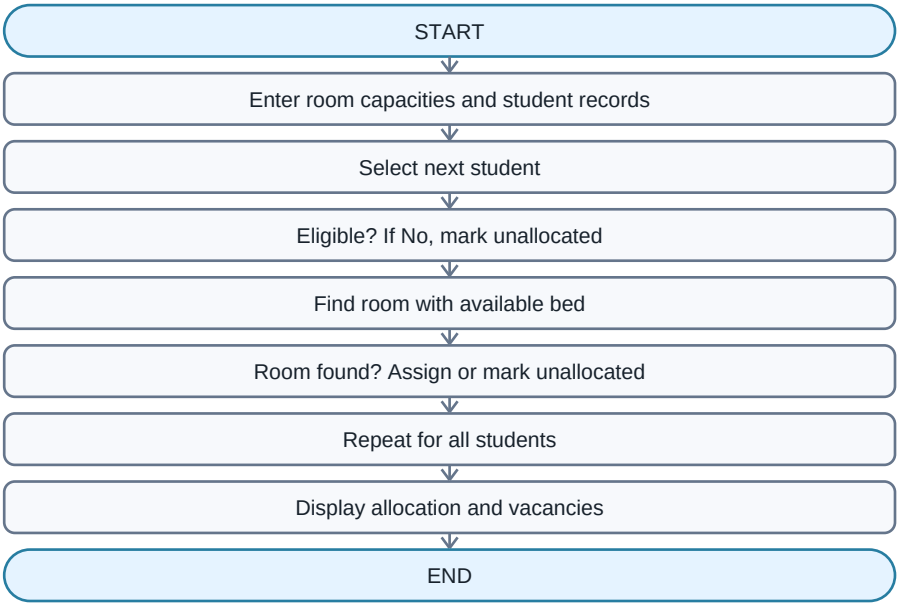
4. For each eligible student, assign the first room with space.

5. Report unallocated students and remaining room capacities.

5. Pseudocode

```
START
Store rooms, capacities, and student records
Create empty allocation list
FOR each student
  IF not eligible, add to unallocated
  ELSE find first room with free capacity
    IF found, assign student and reduce capacity
    ELSE add student to unallocated
DISPLAY room assignments, vacancies, unallocated list
END
```

6. Flowchart



7. Python Source Code

```
rooms = [("A-101", 2), ("A-102", 1), ("B-201", 2)]
```

```
students = [
    ("Asha", True),
    ("Bala", True),
    ("Charan", False),
    ("Divya", True),
    ("Eshan", True),
    ("Farah", True),
    ("Gita", True)
]

remaining = {}
allocation = {}
for room, capacity in rooms:
    remaining[room] = capacity
    allocation[room] = []

unallocated = []
for name, eligible in students:
    assigned = False
    if eligible:
        for room, capacity in rooms:
            if remaining[room] > 0:
                allocation[room].append(name)
                remaining[room] -= 1
                assigned = True
                break
    if not assigned:
        unallocated.append(name)

for room, members in allocation.items():
    print(room, ":", members, "| vacant beds:", remaining[room])
print("Unallocated students:", unallocated)
print("Unallocated count:", len(unallocated))
```

8. Test Cases

Test / Input	Expected result
Asha eligible, A-101 has 2 beds	Assigned to A-101 if first vacancy
Charan not eligible	Unallocated
7 students, 5 beds total, 1 ineligible	Exactly 2 unallocated in this sample

9. Sample Output

```
A-101 : ['Asha', 'Bala'] | vacant beds: 0
A-102 : ['Divya'] | vacant beds: 0
B-201 : ['Eshan', 'Farah'] | vacant beds: 0
Unallocated students: ['Charan', 'Gita']
Unallocated count: 2
```

10. Conclusion

The program demonstrates capacity-based hostel allocation and reports students who remain unallocated. A production system would require more detailed room eligibility, preference, and grouping rules.

Final Submission Checklist

■ Fill in your name, roll number, section, college, and academic year on the cover page.

■ Check every program against the exact rules in your assignment sheet.

■ Run each Python program in your own Python environment and capture genuine output screenshots.

■ If the faculty requires separate .py files, save each code listing as its own file with a meaningful name.

■ Save flowcharts separately as image files if required, even though flowcharts are included in this PDF.

■ Upload the required files to your own GitHub repository named Python-Activity-Based-Learning, as stated in the assignment.

■ Review the report and make sure sample outputs match the final code you submit.

End of sample documentation